

ICS - 344

DVSA Security Project

Vulnerability Discovery & Remediation in Serverless AWS Environments

Team Members: Abdullah Alzahrani · Moayd Shahat · Osama Alghamdi · Rami Almathani

Course:

ICS-344-Information & Network Security

Semester:

Spring 2025 · KFUPM

Project Overview

What is DVSA?

Damn Vulnerable Serverless Application

An intentionally insecure serverless app designed for security testing and education.

- Hosted on AWS
- Simulates real-world serverless attack surfaces
- Based on OWASP Serverless Top 10

AWS Services Used:

S3
Storage

API GW
Routing

Lambda
Functions

DynamoDB
Database

Cognito
Auth

CloudWatch
Logging

Project Goal



Identify

Discover serverless vulnerabilities



Exploit

Reproduce attacks with evidence



Fix

Apply secure mitigations



Verify

Confirm fixes with post-tests



Document

Record lessons learned

Methodology — How We Approached Each Vulnerability

01 Identify

Review source code, configs, and AWS resources to locate the vulnerability

02 Exploit

Craft a payload or attack scenario to reproduce the vulnerability reliably

03 Evidence

Capture screenshots, CloudWatch logs, and HTTP responses as proof

04 Mitigate

Apply code fixes, IAM changes, or AWS configuration hardening

05 Verify

Re-run the attack to confirm the fix and observe secure behavior

06 Learn

Summarize root cause, impact, and takeaways per OWASP Serverless Top 10

Vulnerability Index — 11 Lessons Covered

L01

Event Injection

OWASP SL1

L02

Broken Authentication

OWASP SL2

L03

Sensitive Data Exposure

OWASP SL3

L04

Insecure Cloud Configuration

OWASP SL4

L05

Broken Access Control

OWASP SL5

L06

Denial of Service

OWASP SL6

L07

Over-Privileged Function

OWASP SL7

L08

Logic Vulnerability

OWASP SL8

L09

Vulnerable Dependencies

OWASP SL9

L10

Unhandled Exceptions

OWASP SL10

L11

Admin Broken Access (Bonus)

Bonus

Affected Component

DVSA-ORDER-MANAGER Lambda

Security Impact

Remote code execution inside a Lambda function. Full server-side compromise of the order processing function.

Root Cause

Unsafe deserialization via node-serializer allowed attacker-controlled JS to be passed in the action field and executed inside Lambda.

Fix / Mitigation

Replace unsafe deserialization with JSON.parse. Validate allowed action values. Reject malformed or unexpected input.

Exploitation

Injected JavaScript in the action field executed arbitrary code. CloudWatch logs confirmed code execution output.

Verification Result

Injected payload no longer executes. Lambda returns a validation error. CloudWatch shows no unauthorized execution.

Affected Component

JWT Token Verification — Backend API

Security Impact

Full account takeover. Any authenticated user could impersonate any other user including admins.

Root Cause

The backend decoded JWT payloads without verifying the token signature, trusting attacker-modified claims.

Fix / Mitigation

Verify JWT signatures using trusted Cognito public keys. Validate all claims (iss, exp, sub) before trusting identity.

Exploitation

Modified the username and sub fields in a JWT payload (without re-signing). Backend accepted the forged token and granted access to another user's account.

Verification Result

Forged JWTs now rejected with 401 Unauthorized. Signature verification confirmed with valid and invalid tokens.

Affected Component

DVSA-ADMIN-GET-RECEIPT Lambda

Security Impact

Unauthorized access to all customer receipts. Sensitive financial and personal data exposed without admin privileges.

Root Cause

The Lambda function generated pre-signed S3 receipt archive URLs without first checking if the requester had admin authorization.

Fix / Mitigation

Add function-level admin authorization check. Protect API routing to admin endpoints. Restrict invocation permissions. Reduce pre-signed URL expiration time.

Exploitation

A regular authenticated user called the admin receipt endpoint directly. The function returned a valid pre-signed S3 URL, exposing all customer receipts.

Verification Result

Non-admin requests now return 403 Forbidden. Pre-signed URL expiry reduced. Receipt data no longer accessible to regular users.

Affected Component

S3 Receipts Bucket + DVSA-SEND-RECEIPT-EMAIL

Security Impact

Bypasses API authentication entirely. Attacker-controlled content triggers backend Lambda. Could be used for phishing or resource abuse.

Root Cause

The S3 receipts bucket allowed direct uploads and lacked proper bucket controls, enabling unauthenticated or unauthorized file writes.

Fix / Mitigation

Enable Block Public Access on the S3 bucket. Add a restrictive bucket policy. Validate and sanitize S3 event input inside Lambda.

Exploitation

Uploaded a crafted file directly to the S3 bucket, bypassing the normal API and authentication flow. This triggered DVSA-SEND-RECEIPT-EMAIL Lambda directly via an S3 event.

Verification Result

Direct upload to S3 now rejected with 403. Lambda validates event source and rejects unexpected input patterns.

Affected Component

Order Update Backend — Authenticated Endpoints

Security Impact

Users can bypass payment and fulfill orders for free. Financial loss and business logic violation.

Root Cause

No authorization check was performed before allowing an authenticated user to invoke privileged order-update functionality.

Fix / Mitigation

Enforce authorization checks on all state-changing endpoints. Validate user role before order updates. Enforce conditional state transitions.

Exploitation

As a regular user, triggered the backend order-update function and set the order status to paid/completed without going through shipping and billing.

Verification Result

Regular users receive 403 on privileged order updates. Order status changes now require proper role and workflow completion.

Affected Component

API Gateway — Billing Endpoint

Security Impact

Service unavailability for all users. Lambda concurrency exhaustion leads to cascading failures across the application.

Root Cause

The billing endpoint had no rate limiting or throttling configured, allowing unlimited concurrent Lambda invocations.

Fix / Mitigation

Configure API Gateway usage plans with rate and burst limits. Set per-endpoint throttling. Monitor with CloudWatch alarms.

Exploitation

Sent many concurrent billing requests, triggering multiple Lambda instances simultaneously. Lambda concurrency exhausted, blocking legitimate users.

Verification Result

Excess requests now return HTTP 429 Too Many Requests. Normal traffic unaffected. CloudWatch confirms throttling behavior.

Affected Component

Lambda IAM Execution Role

Security Impact

A single compromised function could pivot to access all application resources. Violates the principle of least privilege.

Root Cause

The Lambda execution role was granted overly broad IAM permissions across S3, DynamoDB, SES, and STS, far beyond what the function needed.

Fix / Mitigation

Remove broad managed policies. Create a custom policy allowing only required actions: specific SES, DynamoDB table access, and CloudWatch logging.

Exploitation

Reviewed the Lambda role's IAM policy. If the function were compromised, an attacker could access all S3 buckets, read/write all DynamoDB tables, send emails, and assume other roles.

Verification Result

Lambda role now has minimal permissions. Unauthorized resource access attempts return `AccessDeniedException`.

Affected Component

Billing + Order-Update Lambda Functions

Security Impact

Customers pay for one item but receive a different or additional item. Financial inconsistency and potential fraud vector.

Root Cause

No concurrency guard or state validation prevented billing and order-update requests from running simultaneously.

Fix / Mitigation

Validate `orderStatus` before processing. Use DynamoDB `ConditionExpression` to block modifications once billing has started.

Exploitation

Sent billing and order-update requests concurrently. Due to the race condition, the final order state differed from what was billed — items were added after billing.

Verification Result

Concurrent requests no longer produce inconsistent state. DynamoDB conditional writes reject conflicting updates.

Affected Component

DVSA-ORDER-MANAGER — node-serializer 0.0.4

Security Impact

Remote code execution via dependency vulnerability. Attacker can run arbitrary code inside the Lambda function.

Root Cause

The function used node-serializer 0.0.4, a package known to execute JavaScript functions when deserializing attacker-controlled input.

Fix / Mitigation

Remove node-serializer dependency. Replace `serialize.unserialize` calls with `JSON.parse`. Integrate dependency scanning in CI/CD.

Exploitation

Passed a serialized object with an embedded JavaScript function as the request payload. The deserializer executed the function inside the Lambda environment.

Verification Result

Payload no longer executes. `JSON.parse` safely rejects executable content. Dependency scanner flags vulnerable packages.

Affected Component

Multiple Lambda Functions (Python handlers)

Security Impact

Information disclosure enabling attackers to map internal code structure, file paths, and error-prone inputs for deeper exploitation.

Root Cause

Lambda handlers did not validate input or catch exceptions, causing raw Python stack traces to be returned to API clients.

Fix / Mitigation

Validate input with `event.get()` with defaults. Wrap handlers in `try/except`. Log full details to CloudWatch only. Return generic error messages to clients.

Exploitation

Sent malformed or missing parameters to several Lambda endpoints. Responses included Python stack traces, file paths, line numbers, and `KeyError` details.

Verification Result

Malformed requests now return generic 400/500 responses. Internal details no longer exposed. Stack traces appear only in CloudWatch.

Affected Component

Order DynamoDB Table — Admin Update Path

Security Impact

Free order fulfillment by bypassing payment. Complete compromise of the order integrity and financial logic.

Root Cause

The backend allowed direct DynamoDB orderStatus updates without enforcing valid state transitions or verifying the requester's authorization.

Fix / Mitigation

Enforce valid state transitions using DynamoDB ConditionExpression. Add authorization checks. Ensure only trusted billing logic can set paid status.

Exploitation

Directly updated an order's status to 'paid/delivered' in DynamoDB without going through the checkout or payment process.

Verification Result

Direct status manipulation blocked by ConditionExpression. Unauthorized updates return ConditionalCheckFailedException.

Key Lessons Learned

10 Principles for Secure Serverless Development

01

Never trust user-controlled input — validate and sanitize everything

02

Always verify authentication tokens — check signatures, not just payloads

03

Enforce authorization at every layer — API, Lambda, and data

04

Apply least privilege to all IAM roles — functions need minimal permissions

05

Harden cloud resources, not only code — bucket policies, ACLs, and configs

06

Add rate limiting to serverless APIs — prevent concurrency exhaustion

07

Validate state before modification — use conditional database writes

08

Scan dependencies regularly — remove vulnerable packages from CI/CD

09

Handle errors safely — log details internally, return generic responses

10

Protect data-layer operations — conditional checks for critical fields